COMP 452 Assignment 3, Part 1 (Minimax Connect 4) Description File

Bryce Dixon (3397649) Feb 2026

TABLE OF CONTENTS

PART 1: Logic and Game Design

CLASS STRUCTURE OVERVIEW

- Main
    - GameBoardBuilder
        - GameBoardCell
    - GamePlayManager
        - GameBoardColumn
        - GamePieceAnimation
        - ComputerPlayer
        - MinimaxGameBoard

HIGH LEVEL GAME LOGIC

- The game is a turn-based loop where a user and the computer alternate turns placing their chips.
    - On the user's turn they select a column (with a click) and place their chip, which triggers an animation of their chip dropping into place. This also triggers for the computer to now take its turn.
    - The computer uses the minimax algorithm with alpha-beta pruning to select its next move. This algorithm is depth limited to 4 and also accounts for terminal conditions within its search (either the computer or user winning, or a full board). Once it takes its move, an animation shows the computer's chip dropping into place and the user is passed control and can take their next turn.
- After each turn the game checks if the last move resulted in the player winning, OR if the last move resulted in the board becoming full.
- The game continues until one of these terminal conditions are met, upon which a game over screen displays and announces the winner.

UI LOGIC (Main.java)

- The main pane (root) is a StackPane, allowing me to stack Panes on top of one another in a particular order.
    - I then add 3 Pane layers to the root:
        - BoardLayer: This is the lowest layer, which displays the game board.
        - AnimationLayer: This is the middle layer and displays the animation of the player's piece falling down the column.
        - OverlayLayer: This is the highest layer and holds overlays that highlight each board column as the user hovers over them.
            - This also holds the game over screen which displays when the game is over.
- This root StackPane is added to the game scene, which is then displayed on the game stage. Altogether this allows for the UI to be separated in its design for easy modular modification and management.

GRID CREATION LOGIC (GameBoardBuilder.java)

- The grid is created by creating a 2D array of GridCell objects (GameBoardCell.java).
- Each gridcell object holds a Rectangle (to make it viewable), a circle (which can be made visible to look like a game piece) and a CellState, which is initialized to empty.
    - CellState is an enum object which can be EMPTY, USER, or COMPUTER.
    - This represents the possible states that each cell can be in and used to display the type of game piece on the board.
- Each object is placed in specific x and y values to produce a grid in a nested for loop.

USER TURN LOGIC (GamePlayManager.java)

- The game board has clickable columns which overlay each column of the game board (GameBoardColumn.java).
- When it is the user's turn and they click on a column, this object detects which column was clicked, which runs a method that does the following:
    - Finds the next available spot in that column by looping through the column from the bottom.
        - If it is the last spot of that column, it marks the column as full.
    - Then triggers a TranslateTransition animation to occur on that column (GamePieceAnimation.java).
    - This animation has a start point above that column and an end point at the next open cell.
    - When the animation ends, it triggers an event which sets the target cell to hide the animation image (the chip), then displays a game piece in the target cell (i.e. turn the cell's circle to the correct colour), and then calls a method to switch turns.
- The system does not allow a column which is full to be clicked on.
- It also does not allow the user to make any plays while the animation is running.
- When the computer takes its turn, it uses the same function + animation as described above.

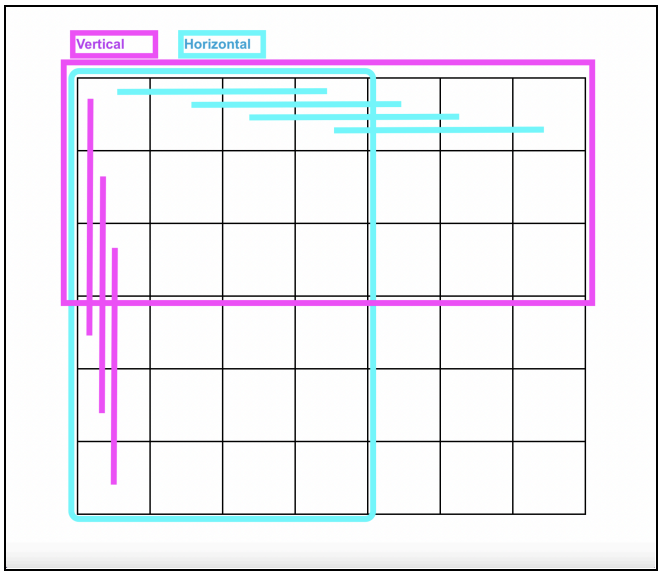
    - This way the user cannot take their turn until the computer has completed its turn.

COMPUTER TURN LOGIC (ComputerPlayer.java)

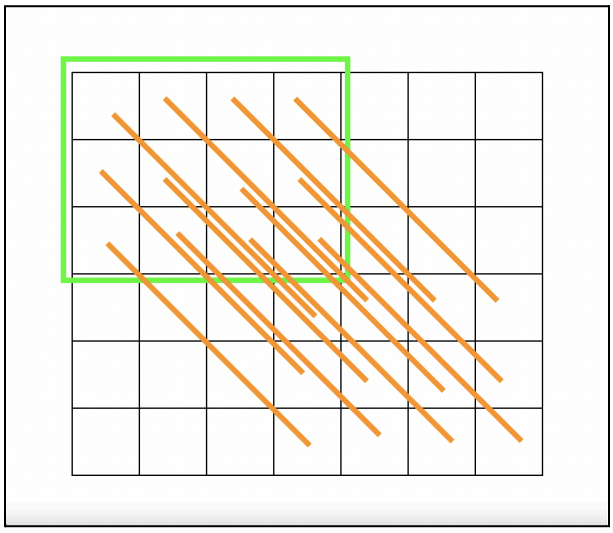
- FIRST STEP:
    - When it is the computer's turn, the first function that is called performs the first maximizing round of the minimax algorithm.
        - This initializes alpha (to a min value) and beta (to a max value).
        - Then gathers all the children from the current board state. This board is created by maintaining a separate game board that holds the state of each cell (MinimaxGameBoard.java) and is updated with each turn in the main game manager.
        - Next begins the recursive call to the minimax algorithm for each child. Each child will start in the minimizing state (as they are the children of the maximizing state)
        - The children for each call are returned in an array where their index in the array matches the column of the move they represent. This way the information received from the minimax recursion can be used to tell which column the computer should take as its next move.
- MINMAX ITERATIONS:
    - The minimax has a maximum search depth of 4, which decrements every time a recursive call to minimax is completed.
        - A depth of 4 was simply chosen as it allowed for 2 moves from each player to be accounted for which is enough to have a notable change in the evaluation function. It was also the value mentioned in the assignment. I have not tested other depths with this program.
    - The algorithm has initial checks for terminal conditions, which are: if we are at the search depth, if we are at a leaf node (i.e. the current game board has either the computer or player winning, or if the game board is full).
        - At this point the recursion ends, the score of the board is returned, and the bubbling up of the values occurs.
        - To check if either player has won, it uses the same function as described below in "GAME OVER CONDITIONS LOGIC".
    - A boolean is used to ensure the recursive searches alternate between maximizing and minimizing.
    - The children for each board state are obtained by cloning the state gameboard (a deep copy as to not alter the original) for each column where there is a move available. This clone is stored in an array at the index which matches the column where it had a move played in.
        - To ensure that the children created matched if we were in the maximizing or minimizing state, the boolean used to determine this in minimax was also passed to the function that created the children boards. This way, when maximizing, the children game boards used computer-pieces when playing a turn, and user-pieces were used for this when minimizing.

    - When Maximizing in Minimax:
        - We take the maximum value received from each recursive call to minimax and set this as the alpha value.
        - Each recursive call has the boolean swapped to ensure the next call is for minimizing.
        - The beta and alpha values are compared to avoid redundant searches and early pruning of subtrees (beta <= alpha).
        - This returns the maximum value received.
    - When Minimizing in Minimax:
        - We take the minimum value received from each recursive call to minimax and set this as the beta value.
        - Each recursive call has the boolean swapped to ensure the next call is for maximizing.
        - The beta and alpha values are compared to avoid redundant searches and early pruning of subtrees (beta <= alpha).
        - This returns the minimum value received.
- Once the desired column is found, it is fed into the turn function, which plays an animation of the computer's chip dropping into place, and passes control to the user to take their next turn.

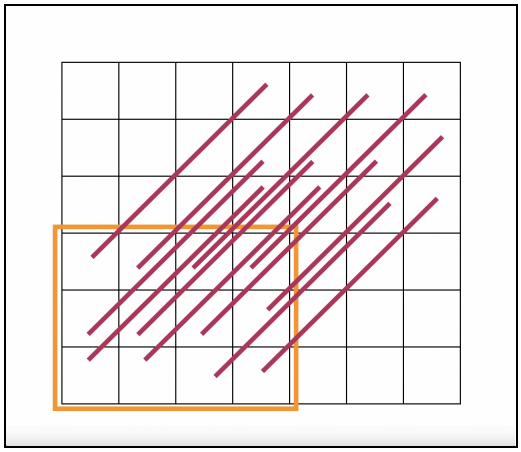
- COMPUTER MINIMAX EVALUATION FUNCTION:
    - For the evaluation function, I decided to go with checking all possible 4 square patterns in the game. This way I could check for a game board which has, or almost has, a winner and base points off of that. Initially I was going to loop through my board in the same way I determine if there is a winner (described below in "GAME OVER CONDITIONS LOGIC"), but this leads to a lot of repeated counting, which does not work for scoring. From this I wanted to only read the potential winning combinations once.
    - I determined what these patterns were by drawing them out (diagrams included below):
        - **Horizontal** patterns (blue in image below) would be counting 4 columns over starting from columns 0,1,2,3 for every row. This avoids going out of bounds and checks all possible combinations.
        - **Vertical** patterns (pink in image below), similar to horizontal, are counting 4 rows down, starting from rows 0,1,2, for all columns. This avoids going out of bounds, but covers all possible combinations.



- **Diagonal pattern 1 (down and to the right)**: if you start from the top left, and move 4 spaces diagonally down from any cell within the range: rows 0-2 and columns 0-3, you get all possible diagonal combinations in this direction (as seen in the green box in the image below).



- **Diagonal pattern 2 (up and to the right)**: If you start from the bottom left and move 4 spaces diagonally up from any cell within the range: rows 3-5 and columns 0-3, you get all possible diagonal combinations in this direction (as seen in the orange box in the image below).



- Using this, each of these 4 cell combinations were checked for how many computer, user, and empty cells they contained, and this was used to determine the evaluation function scores. Situations which favoured the computer were scored positively, while situations which favoured the user scored negatively. Situations that lead to the computer winning or user winning heavily outweighed all other instances as these are imperative to gameplay. These numbers were chosen based on simplicity and were initially just multiples of 10. I increased the winning score (and losing scores) to 100,000 as these were seemingly not being prioritized by the AI initially and increasing the weighting improved on this.
  - These scores are outlined in the table below:

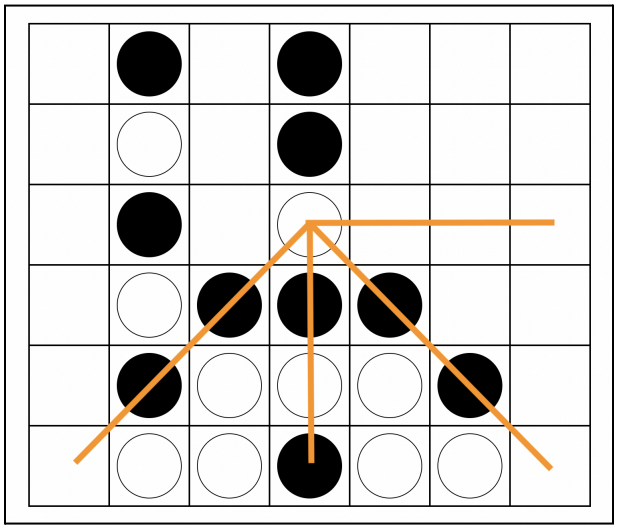
TABLE: Evaluation Function Values applied within the Minimax algorithm

Number Empty cells	Player Type	Number of player chips	Score	Rationale
0	Computer	4	+100,000	The computer winning is just as important as the user winning.
	User	4	-100,000	
1	Computer	3	+1,000	The user nearly winning is more important than the computer nearly winning, so we want to avoid this.
	User	3	-100,000	
2	Computer	2	100	This would mean both have a shot at winning in this space.
	User	2	-100	
3	Computer	1	10	This would

	User	1	-10	mean both have a shot at winning in this space.
All other situations are given a score of 0, as I felt they were neutral.				

GAME OVER CONDITIONS LOGIC

- After each player takes their turn, the game board checks if their move has done 2 things:
    - CHECK IF THAT PLAYER JUST WON:
        - The algorithm loops through each cell in the board (starting from 0,0 or top left) looking for pieces that match the player that just took their turn.
        - When it finds a match, it checks for winning combinations (seen in the image below in orange), which are:
            - 1. Horizontal to the right, 2. vertically down, and diagonal 3. down to the left and 4. down to the right.
        - When checking these spaces:
            - if a space does not match the player we are checking for, it stops the search (as this is clearly not a winning combination).
            - If a space is out of bounds, it stops the search.
        - If a match is found, the game is over and the winner is announced.



    - CHECK IF THE BOARD IS FULL:
        - To check if the board is full, the game simply loops through the column overlay objects mentioned previously and checks if any have space.
            - These overlay objects are marked as "full" using a boolean when the last spot in the column is taken. This is used to turn off the hover overlay for that column to indicate there is no room.

- Using this, if these are all marked as full, then this means the game is tied (as this occurs after the program has checked for a winner).

PART 2: Game Compilation Instructions

IF YOU ARE USING JAVA 8:

- Since JavaFX is included with Java8, you can simply navigate to the directory where you have saved the jar file and run:
    - java -jar A3P1_452_Dixon_3397649.jar

IF YOU ARE USING JAVA 11 and HIGHER:

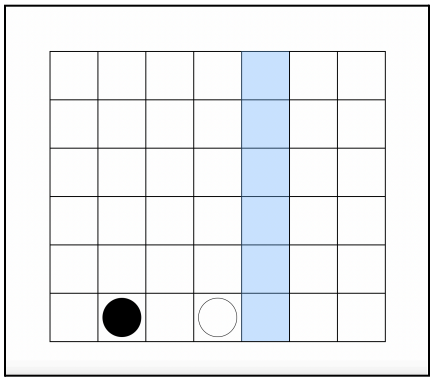
- This project was developed and tested on a MacOS device using Java 17, meaning I needed to download and install JavaFX. I used JavaFX version 21.0.9.
- To run the .jar file, save the file to your device in a known location.
- Ensure JavaFX is downloaded, and find the path to the JavaFX 'lib' folder.
- In your command line interface (CLI), Navigate to the location where you saved the .jar file.
    - Now we tell java what libraries to include from JavaFX, as well as where to find them (i.e. the path to "lib" mentioned above), and then run the program.
    - An example of this is:

```
desktop % java \
--module-path /Users/bd/Desktop/AU/COMP_452/javafx/lib \
--add-modules javafx.controls,javafx.graphics \
-jar A3P1_452_Dixon_3397649.jar
```

        - Where the file (A3P1_452_Dixon_3397649.jar) is stored on on my desktop, the path to my javaFX library is `/Users/bd/Desktop/AU/COMP_452/javafx/lib`, and we are using the modules `javafx.controls,javafx.graphics` from the JavaFX library to run the program.

PART 3: Game Play Instructions

- Hover over the column of interest, you will see it light up in blue if it is available (seen in image below). If a column does not light up when hovered over, it is full and cannot hold anymore chips.
- Click on that column to place the chip, you will see the chip drop into place in that column.
- The goal of the game is to get 4 chips in a row in any direction (horizontal, vertical or diagonal) before the computer does.
- Refer to this link for detailed Connect4 game instructions if needed: https://instructions.hasbro.com/en-my/instruction/connect-4-game

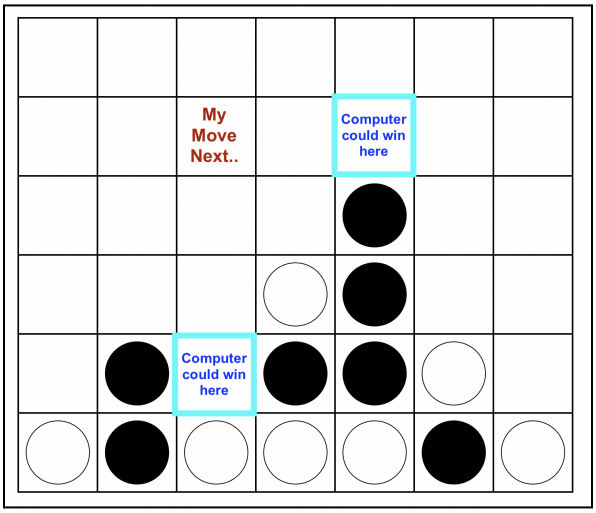


PART 4: Current Game Bugs/ Problems

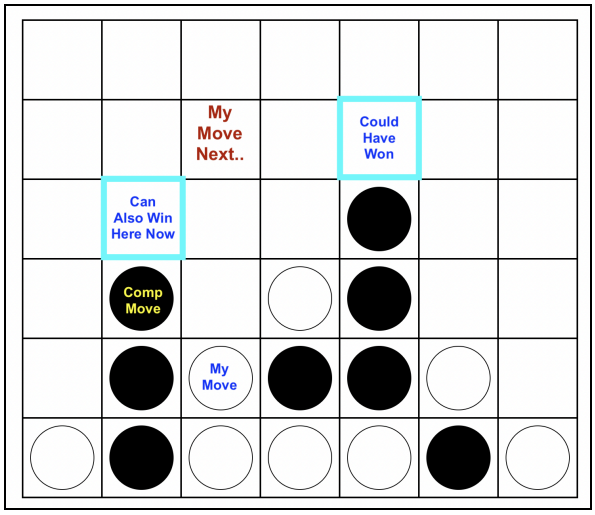
- PROBLEMS/ LIMITATIONS:
    - The computer player logic for the game works well, as the computer player is tricky to beat, but one thing that could be done to improve it would be in the evaluation function:
        - The algorithm checks all possible 4 cell combinations and looks if there is blank space beside the user or computer chip cells. However, it does not check if these empty cells are playable in the next turn, so it is possible that the computer may prioritize an empty cell on a diagonal that it will not be able to play at in the immediate future.
        - This could be improved by simply checking the empty cells and seeing if they are at the bottom of their column or not and if they are, they are playable and thus could be prioritized more than non-playable empty cells.
        - This is noticeable in some of the computer's moves throughout gameplay, but overall the game does a good job of blocking the user from winning.
        - Although, one thing to note with this is due to the computer seeing all possible orientations, it often blocks the user's moves which may be planned far in advance, which is a benefit of it not evaluating only the

playable moves. As this way it is anticipating beyond its depth in a way. This appears to be noticeable when playing, as the computer often blocks the user's moves well in advance.

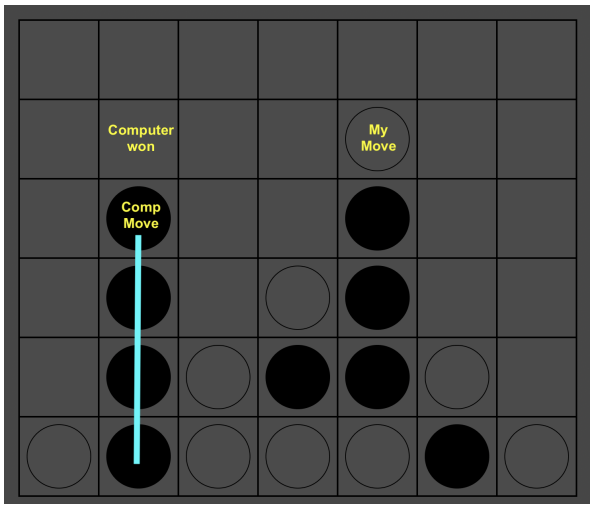
- I did not test other depths for my minimax algorithm, and this is something that could be altered to potentially yield better results.
- The animations for the game are representative of play, but a small bounce animation could be added when the chip hits the bottom of the column.
- To restart the game, you need to re-run the code, which can break up game play a bit for the user.
- There is no audio implemented, which could enhance gameplay if the game was to be built on further.
- When the game is over, it does not highlight the winning cells, which could be a nice addition if the game was to be improved on.
- One thing to note, it is not an issue, more of the computer's ability to look ahead. Sometimes the computer will have a clear win, and it will not take it. But then a few moves later you will see it was solidifying its win in multiple ways, so that it is impossible for the user to win. I will show example images of this below:



← 1. Computer could win in two places.



← 2. I block one of them, and the computer adds another spot where they could win instead of winning.



← 3. computer finally decides to win.

- BUGS:
    - I have not run into any known crashes or performance issues when testing gameplay.